

You said:

```
@GetMapping("/loan") public String createLoan(@RequestParam(required = false, name = "userQ") String userQ, @RequestParam(required = false, name = "userId") Long userId, @RequestParam(required = false, name = "copyCode") Long copyId, Model model) { model.addAttribute("userQ", userQ); model.addAttribute("users", (userQ == null || userQ.isBlank()) ? List.of() : loanService.searchUsers(userQ).stream().map(UserSummaryDto::from).toList()); if(userId != null) { model.addAttribute("selectedUser", UserSummaryDto.from(userRepository.findById(userId).orElseThrow(() -> new NotFoundException("User not found.")))); } if(copyId != null){ model.addAttribute("copy", CopyDetailsDto.from(copyRepository.findById(copyId).orElseThrow(() -> new NotFoundException("User not found.")))); if(reservationRepository.findByIdAndCopyId(userId, copyId).isEmpty()) model.addAttribute("reservedForUser", false); else model.addAttribute("reservedForUser", true); } return "admin-loan"; } como mejorarias eso
```

ChatGPT said:

Puedes mejorarlo bastante en **legibilidad, robustez y evitar llamadas/errores innecesarios**. Ahora mismo hay 3 “olores” claros:

1. **userId puede ser null** pero lo usas dentro del bloque `copyId != null` al consultar `reservationRepository.findByIdAndCopyId(userId, copyId)` → eso te puede romper o hacer una query rara.
2. **Mensaje de error incorrecto**: en el `copyRepository.findById` estás tirando "User not found." (debería ser "Copy not found.").
3. Mucho `if` anidado y repetición. Se puede expresar con `Optional`, booleans claros y helpers.

Aquí tienes una versión más limpia (misma funcionalidad, menos “puntas”):

```
@GetMapping("/loan")
public String createLoan(
    @RequestParam(required = false, name = "userQ") String userQ,
    @RequestParam(required = false, name = "userId") Long userId,
    @RequestParam(required = false, name = "copyCode") Long copyId,
    Model model
) {
    model.addAttribute("userQ", userQ);

    var users = (userQ == null || userQ.isBlank())
        ? List.<UserSummaryDto>of()
```

```

        : loanService.searchUsers(userQ).stream()
            .map(UserSummaryDto::from)
            .toList();
model.addAttribute("users", users);

UserSummaryDto selectedUser = null;
if (userId != null) {
    var user = userRepository.findById(userId)
        .orElseThrow(() -> new NotFoundException("User not found."));
    selectedUser = UserSummaryDto.from(user);
    model.addAttribute("selectedUser", selectedUser);
}

if (copyId != null) {
    var copy = copyRepository.findById(copyId)
        .orElseThrow(() -> new NotFoundException("Copy not found."));
    model.addAttribute("copy", CopyDetailsDto.from(copy));

    boolean reservedForUser = (userId != null) &&
        reservationRepository.findByIdAndCopyId(userId,
copyId).isPresent();

    model.addAttribute("reservedForUser", reservedForUser);
}

return "admin-loan";
}

```

Qué mejora esto

- **No consulta reservas si userId es null** (antes podía petar o devolver algo inesperado).
- **Errores correctos** (Copy not found.).
- reservedForUser se calcula en 1 línea y siempre se setea cuando hay copyId.
- Variables con nombre y flujo más fácil de leer.

Extras (si quieres subir nivel)

- Cambia findByIdAndCopyId(...).isEmpty() por un método existsByUserIdAndCopyId(...) en el repo (más eficiente y más claro):

```

boolean reservedForUser = userId != null &&
reservationRepository.existsByUserIdAndCopyId(userId, copyId);

```

- Si esto crece, mueve la “carga de modelo” a métodos privados (addUsers, addSelectedUser, addCopyAndReservationFlag) para que el controller quede minimalista.
- Valida coherencia: si viene copyId pero no userId, quizás tu UI debería tratarlo distinto (mostrar copy pero no “reservedForUser”, o forzar selección de usuario).

Si me dices si copyId puede venir sin userId (por navegación de la UI), te lo dejo aún más redondo con esa casuística.